جامعة الإمام عبدالرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

College of Computer Science and Information Technology

تامولعملا ةيقنو بساحلا مولع
ةيلك

DEPARTMENT OF COMPUTER ENGINEERING

ARTI 404 – Image Processing

STUDENT PROCEDURAL MANUAL
Lab 4

2025 - 2026

College	Department	Course
College of Computer Science and Information Technology	Department of Computer Engineering	ARTI404 – Image Processing

Practical Session Plan

Session Topic/Title	Session No.	Session Duration (Minutes)
Intensity Transformations and Filtering-Spatial Domain	4	100

1- Session Outcomes

Outcome#2: Write a program that implements fundamental image processing algorithms.

2- Tool(s)/Software

- Python 3
- Anaconda
- IDE for Python: Jupyter, Spyder.
- Libraries: OpenCV, NumPy, Pillow, Skimage, Matplotlib.

3- Procedural steps (Tasks)

Task#1: Thresholding

Step#1: Apply thresholding with a fixed threshold on an image:

In image processing, thresholding is a common technique used to separate objects or regions in an image based on their pixel intensities. It involves converting a grayscale or color image into a binary image, where pixels are classified as either "foreground" or "background" based on a specified threshold value.

The threshold value acts as a dividing point, determining which pixels are considered part of the foreground (above the threshold) and which are part of the background (below the threshold). The process of thresholding essentially simplifies the image by reducing the pixel values to two categories, simplifying subsequent analysis or feature extraction. The result of thresholding is a binary image where foreground objects are represented by white pixels and background areas are represented by black pixels.

Thresholding can be used for various image processing tasks, such as image segmentation, object detection, and feature extraction. Different thresholding techniques and algorithms exist to handle specific scenarios, such as adaptive thresholding, where the threshold value varies across different regions of the image.

It's important to note that choosing an appropriate threshold value can be challenging, as it depends on factors like image quality, lighting conditions, and the characteristics of the objects or regions of interest. Experimentation and adjusting the threshold value may be necessary to achieve desired results.

```
import cv2

# Load the image in grayscale
img = cv2.imread('../images/Parrot.png', cv2.IMREAD_GRAYSCALE)

# Set the threshold values
threshold_values = [0, 50, 100, 150, 200]

# Apply thresholding with different threshold values
for threshold_value in threshold_values:
    ret, thresh = cv2.threshold(img, threshold_value, 255,
cv2.THRESH_BINARY)
    cv2.imshow(f'Thresholded Image ({threshold_value})',
thresh)

# Display the original image
cv2.imshow('Original Image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Task#2: Histogram Processing

Step#1: Load necessary libraries

```
import matplotlib
import matplotlib.pyplot as plt
import numpy as np

from skimage import data, img_as_float
from skimage import exposure
```

Step#2: Load moon image

```
img = data.moon()
```

Step#3: Rescale intensity values to include all the intensities that fall within the 2nd and 98th percentiles

```
# Contrast stretching
p2, p98 = np.percentile(img, (2, 98))
img_rescale = exposure.rescale_intensity(img, in_range=(p2, p98))
```

Step#4: Display the image with its histogram of (step#2) and (step#3)

```
fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img, cmap = 'gray')
plt.axis('off')
plt.title('Original image')
fig.add_subplot(2, 1, 2)
plt.hist(img.flat, bins = 256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

```
fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale, cmap = 'gray')
plt.axis('off')
plt.title('Contrast stretch')
fig.add_subplot(2, 1, 2)
plt.hist(img_rescale.flat, bins = 256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

4- Assessment

The student will be asked to accomplish the following tasks within the lab time.

Task#1: Using the same 'moon image' Rescale intensity values to include all the intensities that fall within the 3nd and 80th percentiles, and plot the histogram

Task#2: Using the same 'moon image' and the exposure.equalize_hist function, display the image and the histogram of the image after flattening the histogram.

Task#3: Use the rocket (as reference) and chelsea images (from skimage.data) and implement histogram matching

5- Resources

Hands-On Image Processing with Python Book

https://scikit-image.org/docs/stable/auto_examples/color_exposure/plot_equalize.html

https://scikit-image.org/docs/0.24.x/auto_examples/color_exposure/plot_histogram_matching.html

Submission

The students need to prepare the report in a PDF format and submit the GitHub link via Blackboard. Make sure the Lab#3 folder contains the following files.

- Jupyter notebook
- Images used